

71. Une grille : un tableau à deux dimensions

Une carte de jeu, c'est des lignes de cases : `carte[ligne][colonne]`.

1. Ce qu'il faut comprendre

`uint8_t carte[8][12]` est une **grille** de 8 lignes de 12 colonnes : 96 cases. On en désigne une avec **deux** indices : `carte[ligne][colonne]`. En mémoire, les lignes sont rangées **l'une après l'autre** ; le compilateur calcule `ligne × 12 + colonne` pour toi.

On la **remplit** avec deux boucles imbriquées et une condition : une case est un mur si elle est **sur le bord** — première ou dernière ligne, **ou** première ou dernière colonne. Les `||` du chapitre 2 disent exactement cela.

Puis on ajoute quelques murs à la main, et l'on **dessine** avec une seconde double boucle : un # pour 1, un point pour 0.

Séparer **remplir** et **dessiner** est important : la grille est le **monde** du jeu, l'écran n'est qu'une **image** de ce monde. Pour savoir si un héros peut avancer, on demande à `carte[1][c]` — pas à l'écran. C'est ce que font les collisions de tous les jeux en grille : Zelda, Pokémon, Boulder Dash.

À toi : ajoute une porte (un 0) au milieu du mur du bas.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction `ecran\(\)` — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **822 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         287 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← écran() - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← texte() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         272 o ← CopierTuiles - 17 lettres sur 43
"#"            1 o ← texte() - un texte gravé
"."            1 o ← texte() - un texte gravé
"MURS:"        5 o ← texte() - un texte gravé
Lettres gravées : MRSU0123456789.:#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
```

DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 3	63 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
ecran()	× 2	21 o sur place, dans main()	+ AttendreVBlank (16 o)
nombre()	× 1	10 o sur place, dans main()	+ EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : une petite carte de 8 lignes sur 12 colonnes, rangée dans une
// GRILLE (un tableau à deux dimensions), puis dessinée avec des # et des points.
//
// Ce qui est nouveau : carte[8][12]
//   = 8 rangées de 12 cases. carte[l][c] = la case de la ligne l, colonne c.
//   l va de 0 à 7, c va de 0 à 11.

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. UN TABLEAU — CARTE

```
uint8_t carte[8][12];           // 8 × 12 = 96 cases. 1 = un mur, 0 = du sol.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    // Remplir : les bords sont des murs.
    for (uint8_t l = 0; l < 8; l++) {        // chaque ligne l (0 à 7)
        for (uint8_t c = 0; c < 12; c++) {    // chaque colonne c (0 à 11)
            if (l == 0 || l == 7 || c == 0 || c == 11) carte[l][c] = 1;
                                                // première ou dernière ligne, OU première ou
                                                // dernière colonne : c'est un bord → mur (1).
            else carte[l][c] = 0;            // sinon : du sol (0).
        }
    }

    // Quelques murs à la main.
    carte[3][4] = 1;                         // ligne 3, colonne 4
    carte[4][4] = 1;                         // ligne 4, colonne 4
    carte[4][8] = 1;                         // ligne 4, colonne 8

    // Dessiner, et compter les murs.
    uint8_t murs = 0;
    for (uint8_t l = 0; l < 8; l++) {
        for (uint8_t c = 0; c < 12; c++) {
            if (carte[l][c] == 1) {          // un mur ?
                texte(4 + c, 3 + l, "#"); // un # ; « 4 + c » et « 3 + l » placent la carte
                                                // à partir de la colonne 4, ligne 3 de l'écran.
                murs++;
            } else {                         // du sol :
                texte(4 + c, 3 + l, "."); // un point.
            }
        }
    }

    texte(4, 13, "MURS:");
    nombre(10, 13, murs);                   // Les bords : 12 + 12 + 6 + 6 = 36, plus 3 à la main = 39.

    ecran(1);                               // On rallume.

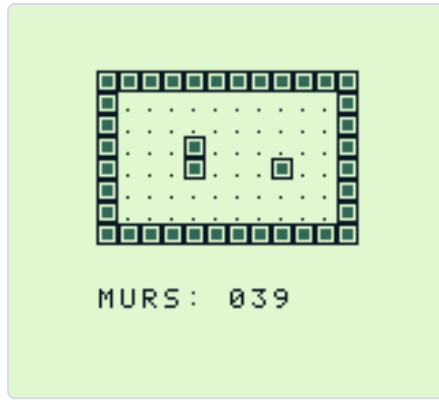
    while (true) {                          // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une salle fermée de 12×8 , trois murs à l'intérieur, et 039 murs comptés.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le mur du haut est plein
- ✓ une ligne du milieu : bord, sol, bord
- ✓ les murs posés à la main sont là
- ✓ 36 de bord et 3 dedans : 39

Ensuite : **71.1. Chapitre 11 — Découper son programme : ce qui vient** — chapitre 11 : Découper son programme. · gameboy3
— écrire une cartouche Game Boy en C++